

Otimização Heurística

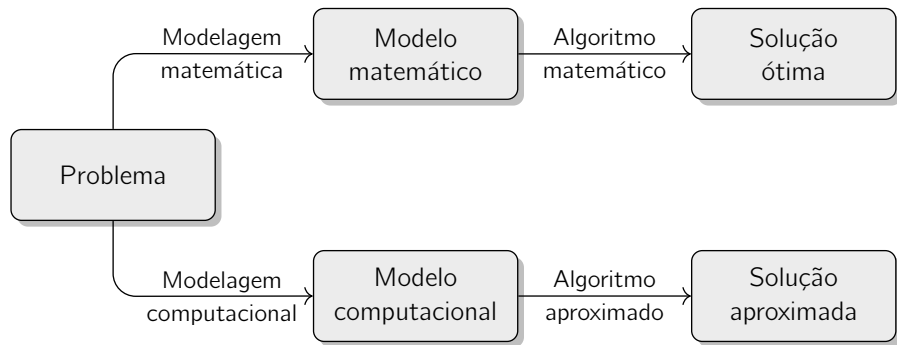
Técnicas e algoritmos

Prof. Marcelo de Souza

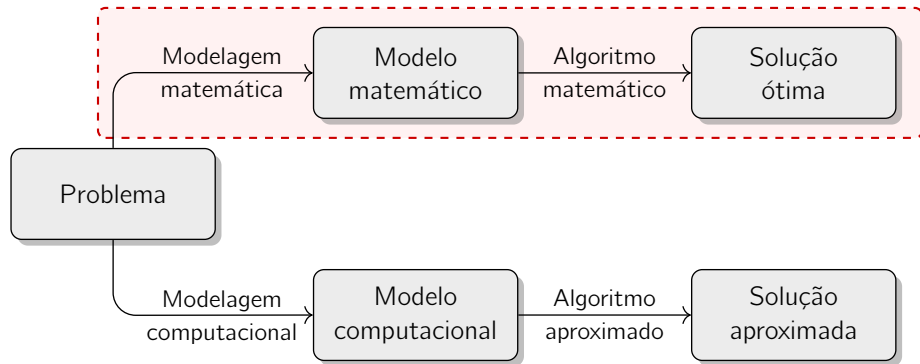
75POT – Pesquisa Operacional e Otimização
Universidade do Estado de Santa Catarina



Etapas, elementos e técnicas da PO



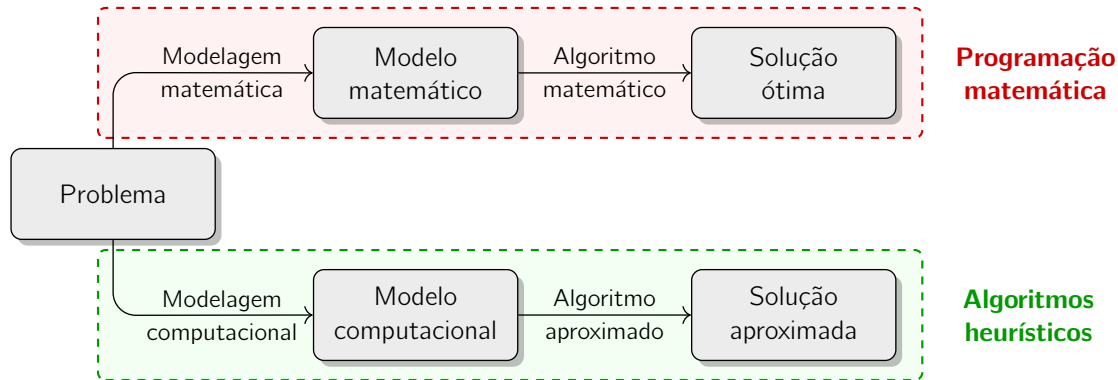
Etapas, elementos e técnicas da PO



**Programação
matemática**

↪ Programação linear, programação inteira ...

Etapas, elementos e técnicas da PO



→ Programação linear, programação inteira ...

→ Buscas por construção, refinamento, recombinação ...



Espaço de soluções

Conjunto de todas as soluções possíveis (candidatas) para uma instância do problema.

- ▶ Também chamado de **espaço de busca**.
- ▶ Depende da **representação** de soluções e pode incluir candidatos **viáveis** e **inviáveis**.



Espaço de soluções

Conjunto de todas as soluções possíveis (candidatas) para uma instância do problema.

- ▶ Também chamado de **espaço de busca**.
- ▶ Depende da **representação** de soluções e pode incluir candidatos **viáveis** e **inviáveis**.

Heurística

Método simples que produz uma solução do espaço de soluções de um problema **específico**.

- ▶ Exemplos: construção (semi-) gulosa, busca local simples.



Espaço de soluções

Conjunto de todas as soluções possíveis (candidatas) para uma instância do problema.

- ▶ Também chamado de **espaço de busca**.
- ▶ Depende da **representação** de soluções e pode incluir candidatos **viáveis** e **inviáveis**.

Heurística

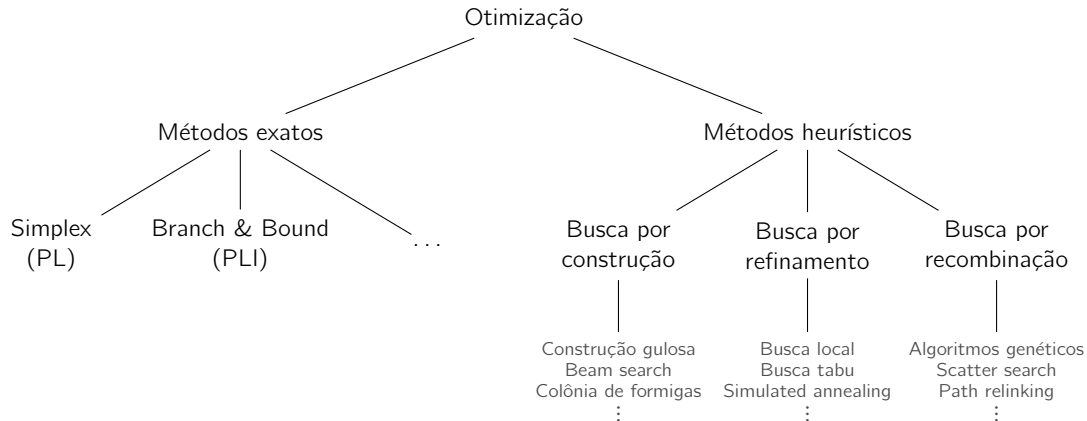
Método simples que produz uma solução do espaço de soluções de um problema **específico**.

- ▶ Exemplos: construção (semi-) gulosa, busca local simples.

Metaheurística

Framework geral e **independente** de problema, que orienta e combina heurísticas para explorar o espaço de busca de forma eficiente.

- ▶ Exemplos: busca local iterada, algoritmos genéticos.





Representação de soluções

Conceito e propriedades

Uma **representação** é a forma como uma solução do problema é **codificada** para que os operadores de busca (construção, refinamento, recombinação) possam manipulá-la.

- ▶ É a decisão de projeto mais **fundamental** para o algoritmo: toda a definição de espaço de busca e operadores depende dela.
- ▶ A mesma instância do problema pode ter **representações diferentes**, cada uma gerando um espaço de busca distinto.
- ▶ Uma boa representação deve ser **completa** (cobrir todas as soluções possíveis), **conexa** (permite alcançar a solução ótima com movimentos a partir de qualquer solução) e, idealmente, **compacta**.

Representação de soluções

Tipos comuns

Binária

Vetor $x \in \{0,1\}^n$ que indica a presença/ausência de um elemento.



Representação de soluções

Tipos comuns

Binária

Vetor $x \in \{0,1\}^n$ que indica a presença/ausência de um elemento.

Permutação

Vetor com uma ordenação de n elementos, sem repetição, quando a solução é uma sequência.



Representação de soluções

Tipos comuns

Binária

Vetor $x \in \{0,1\}^n$ que indica a presença/ausência de um elemento.

Permutação

Vetor com uma ordenação de n elementos, sem repetição, quando a solução é uma sequência.

Valores numéricos

Vetor de números inteiros/reais em que cada posição assume um valor numérico de um domínio.



Representação de soluções

Tipos comuns



Binária

Vetor $x \in \{0,1\}^n$ que indica a presença/ausência de um elemento.

Permutação

Vetor com uma ordenação de n elementos, sem repetição, quando a solução é uma sequência.

Valores numéricos

Vetor de números inteiros/reais em que cada posição assume um valor numérico de um domínio.

Estruturas mais complexas

Árvores, grafos, conjuntos, etc., usadas quando a solução tem características que não são capturadas por abordagens mais simples.

Representação de soluções

Tipos comuns



Binária

Vetor $x \in \{0,1\}^n$ que indica a presença/ausência de um elemento.

Permutação

Vetor com uma ordenação de n elementos, sem repetição, quando a solução é uma sequência.

Valores numéricos

Vetor de números inteiros/reais em que cada posição assume um valor numérico de um domínio.

Estruturas mais complexas

Árvores, grafos, conjuntos, etc., usadas quando a solução tem características que não são capturadas por abordagens mais simples.

Representações indiretas

Conjunto de valores que são usados por um decodificador para então gerar a solução.

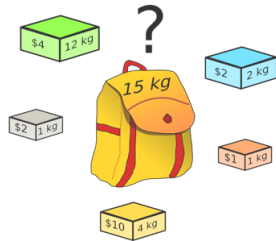


Representação de soluções

Problema da mochila (representação binária)

No **problema da mochila**, temos um conjunto de n itens com valores e pesos, e uma mochila com determinada capacidade. Queremos escolher itens maximizando a soma dos seus valores, sem que a soma dos seus pesos ultrapasse a capacidade da mochila.

Podemos adotar uma **representação binária**, usando um vetor com n posições. Cada posição indica se o elemento correspondente é escolhido.



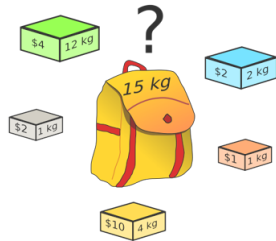
Representação de soluções

Problema da mochila (representação binária)

No **problema da mochila**, temos um conjunto de n itens com valores e pesos, e uma mochila com determinada capacidade. Queremos escolher itens maximizando a soma dos seus valores, sem que a soma dos seus pesos ultrapasse a capacidade da mochila.

Podemos adotar uma **representação binária**, usando um vetor com n posições. Cada posição indica se o elemento correspondente é escolhido.

A solução $S = \{0, 1, 1, 0, 0, 1, 0, 1, 1, 0\}$ indica que os itens $\{2, 3, 6, 8, 9\}$ foram escolhidos.

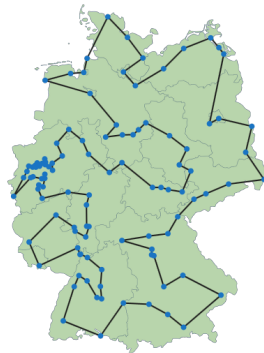


Representação de soluções

Problema do caixeiro viajante (representação por permutação)

O **problema do caixeiro viajante** consiste em, dado um conjunto de n cidades e as distâncias entre cada par delas, encontrar a rota de menor custo total que visita cada cidade exatamente uma vez e retorna à cidade de origem.

Podemos adotar uma **representação por permutação**, usando uma ordenação das n cidades. Essa permutação indica a ordem de visita das cidades.



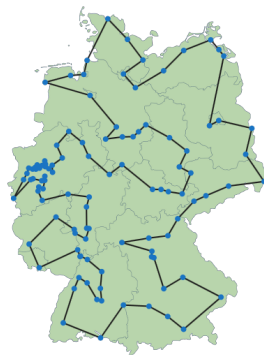
Representação de soluções

Problema do caixeiro viajante (representação por permutação)

O **problema do caixeiro viajante** consiste em, dado um conjunto de n cidades e as distâncias entre cada par delas, encontrar a rota de menor custo total que visita cada cidade exatamente uma vez e retorna à cidade de origem.

Podemos adotar uma **representação por permutação**, usando uma ordenação das n cidades. Essa permutação indica a ordem de visita das cidades.

A solução $S = \{2, 4, 1, 5, 6, 8, 3, 7\}$ indica que a rota inicia na cidade 2, depois visita a cidade 4, e assim sucessivamente até chegar na cidade 7 e então retornar à origem (cidade 2).

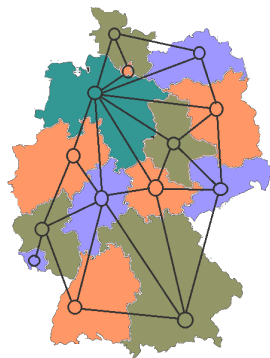


Representação de soluções

Problema da coloração de vértices (representação por vetor inteiro)

O **problema da coloração de vértices** consiste em, dado um grafo com n vértices, atribuir uma cor a cada vértice de forma que vértices adjacentes recebam cores diferentes, usando o menor número possível de cores.

Podemos adotar uma **representação por vetor inteiro** com n posições, onde cada posição indica a cor $c \in \{1, 2, \dots, C\}$ atribuída ao vértice correspondente.



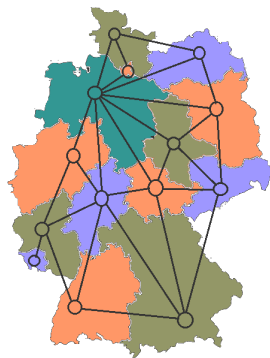
Representação de soluções

Problema da coloração de vértices (representação por vetor inteiro)

O **problema da coloração de vértices** consiste em, dado um grafo com n vértices, atribuir uma cor a cada vértice de forma que vértices adjacentes recebam cores diferentes, usando o menor número possível de cores.

Podemos adotar uma **representação por vetor inteiro** com n posições, onde cada posição indica a cor $c \in \{1, 2, \dots, C\}$ atribuída ao vértice correspondente.

A solução $S = \{1, 1, 2, 1, 3, 3, 2, 1\}$ indica que o grafo está colorido com três cores $\{1, 2, 3\}$. A cor 1 é atribuída aos vértices $\{1, 2, 4, 8\}$, a cor 2 aos vértices $\{3, 7\}$ e a cor 3 aos vértices $\{5, 6\}$.





Implementação

Problema da mochila – classe Instance

```
1  import random
2
3  class Instance:
4      def __init__(self, weights, values, capacity):
5          assert len(weights) == len(values)
6          self.n = len(weights)
7          self.weights = weights
8          self.values = values
9          self.capacity = capacity
10
11  @staticmethod
12  def random_instance(n, capacity_ratio=0.5, seed=None):
13      rng = random.Random(seed)
14      weights = [rng.randint(1, 50) for _ in range(n)]
15      values = [rng.randint(1, 100) for _ in range(n)]
16      capacity = int(sum(weights) * capacity_ratio)
17      return Instance(weights, values, capacity)
```



Implementação

Problema da mochila – classe Solution

```
1 class Solution:
2     def __init__(self, instance, x=None):
3         self.instance = instance
4         self.x = x if x is not None else [0] * instance.n
5
6     def total_weight(self):
7         return sum(self.instance.weights[i] for i in range(self.instance.n) if self.x[i] == 1)
8
9     def total_value(self):
10        return sum(self.instance.values[i] for i in range(self.instance.n) if self.x[i] == 1)
11
12    def is_feasible(self):
13        return self.total_weight() <= self.instance.capacity
14
15    def objective(self):
16        if not self.is_feasible(): return float("-inf")
17        return self.total_value()
```



Tratamento de restrições

Principais estratégias

Rejeição: descarta completamente qualquer solução inviável.

- ▶ Simples e direto.
- ▶ Mas perde-se informação, uma vez que soluções que violam a restrição por muito pouco são tratadas igual a péssimas soluções.
- ▶ Em espaços de busca muito restritos, pode ser difícil até encontrar uma solução viável.

```
1 def objective(self):  
2     if not self.is_feasible():  
3         return float("-inf")  
4     return self.total_value()
```



Tratamento de restrições

Principais estratégias

Penalização: permite soluções inviáveis, mas penaliza proporcionalmente à violação.

- ▶ **Penalização estática:** fator de penalização fixo (código abaixo); simples, mas difícil de calibrar (penalização fraca demais não desincentiva infactibilidade; forte demais empurra a busca só para soluções triviais/vazias).
- ▶ **Penalização dinâmica:** o fator cresce ao longo das iterações (começa permissivo, termina restritivo).
- ▶ **Penalização adaptativa:** o fator se ajusta conforme o histórico recente de soluções encontradas (e.g., aumenta se muitas soluções viáveis seguidas, diminui se muitas inviáveis).

```
1 def objective(self, penalty_factor=10):  
2     excess = max(0, self.total_weight() - self.instance.capacity)  
3     return self.total_value() - penalty_factor * excess
```




Tratamento de restrições

Principais estratégias

Reparo: permite gerar soluções inviáveis, mas aplica um procedimento que as corrige.

- ▶ A busca sempre trabalha com soluções viáveis, sem precisar calibrar penalização.
- ▶ Porém, o reparo introduz um viés (a escolha de como reparar afeta quais soluções são alcançáveis); pode ser custoso computacionalmente se aplicado muitas vezes.

```
1 def repair(self):  
2     while not self.is_feasible():  
3         i = min((i for i in range(self.instance.n) if self.x[i] == 1),  
4                 key=lambda i: self.instance.values[i])  
5         self.x[i] = 0
```



Tratamento de restrições

Principais estratégias

Representação/operações que garantem viabilidade: desenhar a representação e as operações de forma que toda solução gerada já seja viável.

- ▶ Elimina o problema, pois não há necessidade de rejeição, penalização ou reparo.
- ▶ Porém, nem sempre é possível projetar um algoritmo assim. Quando possível, ele pode restringir artificialmente quais soluções a busca consegue alcançar.

Exemplo

Caixeiro viajante: qualquer permutação das n cidades já é uma solução viável, pois define diretamente um circuito Hamiltoniano, sem violar nenhuma restrição do problema.[†]

- ▶ Solução $\pi = (C, A, D, B) \Rightarrow C \rightarrow A \rightarrow D \rightarrow B \rightarrow C$.

[†]No problema da mochila, podemos usar uma representação (indireta) por permutação, onde um decodificador guloso insere os itens na ordem da permutação até a capacidade se esgotar.



Implementação

Problema da mochila – classe `Solution` (melhorias)

É **muito importante** que a implementação seja o mais eficiente possível. Cada chamada a `total_weight()` ou `total_value()` percorre o vetor `x` inteiro [$\mathcal{O}(n)$] por chamada, mesmo que só um item tenha mudado desde a última avaliação.

Avaliação incremental

Manter o peso e o valor atuais como atributos da solução, atualizados em $\mathcal{O}(1)$ a cada flip, em vez de recalculados do zero.

- ▶ Atributos `current_weight` e `current_value` são computados na criação da solução.
- ▶ A cada `flip(i)`, soma-se ou subtrai-se apenas o peso/valor do item `i`.
- ▶ As funções `total_weight`, `total_value`, `is_feasible` e `objective` passam a ser $\mathcal{O}(1)$.



Implementação

Problema da mochila – classe `Solution` (melhorias)

É **muito importante** que a implementação seja o mais eficiente possível. Cada chamada a `total_weight()` ou `total_value()` percorre o vetor `x` inteiro [$\mathcal{O}(n)$] por chamada, mesmo que só um item tenha mudado desde a última avaliação.

Avaliação incremental

Manter o peso e o valor atuais como atributos da solução, atualizados em $\mathcal{O}(1)$ a cada flip, em vez de recalculados do zero.

- ▶ Atributos `current_weight` e `current_value` são computados na criação da solução.
- ▶ A cada `flip(i)`, soma-se ou subtrai-se apenas o peso/valor do item `i`.
- ▶ As funções `total_weight`, `total_value`, `is_feasible` e `objective` passam a ser $\mathcal{O}(1)$.

Na prática: *speedup* de $\sim 2500\times$ em experimentos preliminares.



Implementação

Problema da mochila – classe Solution (melhorias)

```
1 class Solution:
2     def __init__(self, instance, x=None):
3         self.instance = instance
4         self.x = x if x is not None else [0] * instance.n
5         self.current_weight = sum(instance.weights[i] for i in range(instance.n) if self.x[i] == 1)
6         self.current_value = sum(instance.values[i] for i in range(instance.n) if self.x[i] == 1)
7
8     def total_weight(self): return self.current_weight
9
10    def total_value(self): return self.current_value
11
12    def is_feasible(self): return self.total_weight() <= self.instance.capacity
13
14    def objective(self):
15        if not self.is_feasible(): return float("-inf")
16        return self.total_value()
17
18    def flip(self, i):
19        delta = 1 if self.x[i] == 0 else -1
20        self.x[i] = 1 - self.x[i]
21        self.current_weight += delta * self.instance.weights[i]
22        self.current_value += delta * self.instance.values[i]
```

(Meta)heurísticas

Técnicas e conceitos

Na sequência, veremos diferentes heurísticas e metaheurísticas (e mais conceitos):

Construção

1. Heurística construtiva
2. Iterated greedy

» ver

» ver

Refinamento

3. Busca local simples
4. Busca tabu
5. Busca local iterada

» ver

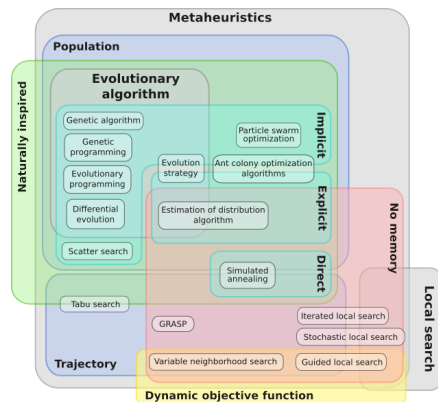
» ver

» ver

Recombinação

6. Scatter search

» ver





Ideia: construir uma solução gradualmente, adicionando elementos de forma incremental.

- ▶ Inicia com uma solução vazia (ou parcial).
- ▶ A cada passo, escolhe um novo elemento e o incorpora à solução, se possível.
- ▶ Termina quando a solução está completa ou nenhum elemento pode ser adicionado.



Heurística construtiva

Ideia: construir uma solução gradualmente, adicionando elementos de forma incremental.

- ▶ Inicia com uma solução vazia (ou parcial).
- ▶ A cada passo, escolhe um novo elemento e o incorpora à solução, se possível.
- ▶ Termina quando a solução está completa ou nenhum elemento pode ser adicionado.

Heurística construtiva

Entrada: estratégia G

Saída: solução s

$s \leftarrow \emptyset$

$C \leftarrow$ elementos candidatos

enquanto $C \neq \emptyset$ e solução não completa **faça**

$c \leftarrow$ seleciona $c \in C$ conforme estratégia G

se c pode ser incorporado a s **então**

$s \leftarrow s \cup \{c\}$

$C \leftarrow C \setminus \{c\}$

retorna s



Elementos candidatos (C)

- ▶ Para o **problema da mochila**: itens ainda não considerados.
- ▶ Para o **caixeiro viajante**: cidades ainda não visitadas.
- ▶ Para a **coloração de vértices**: vértices ainda não coloridos.



Elementos candidatos (C)

- ▶ Para o **problema da mochila**: itens ainda não considerados.
- ▶ Para o **caixeiro viajante**: cidades ainda não visitadas.
- ▶ Para a **coloração de vértices**: vértices ainda não coloridos.

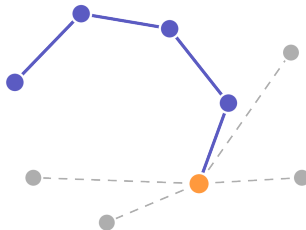
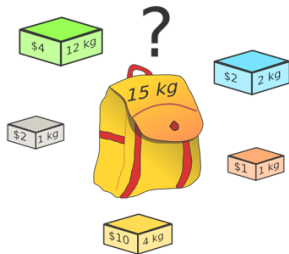
Estratégias de seleção (G)

- ▶ **Aleatória**: escolha um elemento aleatoriamente.
- ▶ **Gulosa**: escolhe o melhor elemento imediato, segundo uma definição de qualidade.
- ▶ **Semi-gulosa**: escolhe aleatoriamente um entre os k melhores elementos.
 - ▶ O valor de k pode ser constante ($k = 10$) ou um percentual dos candidatos ($k = 10\%$).
 - ▶ Caso $k = 1$, trata-se de uma heurística construtiva **gulosa**.
 - ▶ Caso $k = n$, trata-se de uma heurística construtiva **aleatória** (ou caminhada aleatória).



A métrica de **qualidade** dos elementos candidatos pode ser **estática** ou **dinâmica**.

- ▶ **Estática:** calculada uma única vez, não depende da solução parcial.
 - ▶ e.g., valor de um item, no problema da mochila.
- ▶ **Dinâmica:** recalculada a cada passo, pois depende do que já foi inserido na solução.
 - ▶ e.g., distância até a última cidade já visitada, no caixeiro viajante (muda a cada passo).





Instância

- ▶ Capacidade da mochila: $P = 16$
- ▶ Itens (i), Peso (p_i), Valor (v_i):
 - ▶ 1: ($p_1 = 3, v_1 = 10$)
 - ▶ 2: ($p_2 = 1, v_2 = 8$)
 - ▶ 3: ($p_3 = 5, v_3 = 15$)
 - ▶ 4: ($p_4 = 7, v_4 = 22$)
 - ▶ 5: ($p_5 = 5, v_5 = 16$)
 - ▶ 6: ($p_6 = 4, v_6 = 9$)

Estratégia de seleção (G)

- ▶ **Gulosa**: v_i é a qualidade de i .

Candidatos (C)

- ▶ Itens ainda não considerados.



Heurística construtiva

Problema da mochila

Instância

- ▶ Capacidade da mochila: $P = 16$
- ▶ Itens (i), Peso (p_i), Valor (v_i):
 - ▶ 1: ($p_1 = 3, v_1 = 10$)
 - ▶ 2: ($p_2 = 1, v_2 = 8$)
 - ▶ 3: ($p_3 = 5, v_3 = 15$)
 - ▶ 4: ($p_4 = 7, v_4 = 22$)
 - ▶ 5: ($p_5 = 5, v_5 = 16$)
 - ▶ 6: ($p_6 = 4, v_6 = 9$)

Estratégia de seleção (G)

- ▶ **Gulosa**: v_i é a qualidade de i .

Candidatos (C)

- ▶ Itens ainda não considerados.

Execução

Passo	Escolhido	Peso	Solução	Aceita?	Valor
Passo 0	–	0	$s = \{\}$	✓	0
Passo 1	Item 4	7	$s = \{4\}$	✓	22
Passo 2	Item 5	12	$s = \{4, 5\}$	✓	38
Passo 3	Item 3	17	$s = \{4, 5, 3\}$	✗	–
Passo 4	Item 1	15	$s = \{4, 5, 1\}$	✓	48
Passo 5	Item 6	19	$s = \{4, 5, 1, 6\}$	✗	–
Passo 6	Item 2	16	$s = \{4, 5, 1, 2\}$	✓	56
Fim	Solução: $s = \{1, 2, 4, 5\}$, valor: 56, peso: 16				



Heurística construtiva

Problema da mochila

Instância

- ▶ Capacidade da mochila: $P = 16$
- ▶ Itens (i), Peso (p_i), Valor (v_i):
 - ▶ 1: ($p_1 = 3, v_1 = 10$)
 - ▶ 2: ($p_2 = 1, v_2 = 8$)
 - ▶ 3: ($p_3 = 5, v_3 = 15$)
 - ▶ 4: ($p_4 = 7, v_4 = 22$)
 - ▶ 5: ($p_5 = 5, v_5 = 16$)
 - ▶ 6: ($p_6 = 4, v_6 = 9$)

Estratégia de seleção (G)

- ▶ **Gulosa**: v_i é a qualidade de i .

Candidatos (C)

- ▶ Itens ainda não considerados.

Execução

Passo	Escolhido	Peso	Solução	Aceita?	Valor
Passo 0	–	0	$s = \{\}$	✓	0
Passo 1	Item 4	7	$s = \{4\}$	✓	22
Passo 2	Item 5	12	$s = \{4, 5\}$	✓	38
Passo 3	Item 3	17	$s = \{4, 5, 3\}$	✗	–
Passo 4	Item 1	15	$s = \{4, 5, 1\}$	✓	48
Passo 5	Item 6	19	$s = \{4, 5, 1, 6\}$	✗	–
Passo 6	Item 2	16	$s = \{4, 5, 1, 2\}$	✓	56
Fim	Solução: $s = \{1, 2, 4, 5\}$, valor: 56, peso: 16				

Alternativas:

- ▶ Usar qualidade como menor p_i ou maior v_i/p_i .
- ▶ Usar uma estratégia semi-gulosa ($k = 3$) ou aleatória.



Heurística construtiva

Implementação – problema da mochila

```
1 def candidates(instance, solution):
2     remaining = instance.capacity - solution.total_weight()
3     return [
4         i for i in range(instance.n)
5         if solution.x[i] == 0 and instance.weights[i] <= remaining
6     ]
7
8 def semi_greedy_construction(instance, k=3, seed=None):
9     rng = random.Random(seed)
10    solution = Solution(instance)
11    while True:
12        cand = candidates(instance, solution)
13        if not cand: break
14        cand.sort(key=lambda i: instance.values[i], reverse=True)
15        rcl = cand[:k]
16        chosen = rng.choice(rcl)
17        solution.flip(chosen)
18    return solution
```



Heurística construtiva

Implementação – problema da mochila (melhorias)

É **muito importante** que a implementação seja o mais eficiente possível. Abaixo são apresentadas algumas melhorias de eficiência da implementação.

Avaliação incremental

Manter o peso atual em uma variável, atualizada em $\mathcal{O}(1)$ a cada escolha, em vez de somar o vetor inteiro a cada chamada.

Ordenar os candidatos uma única vez

A ordem de qualidade entre os itens não muda, só o conjunto de candidatos que encolhe. Basta ordenar no início $[\mathcal{O}(n \log n)]$, não a cada passo.

Poda permanente de itens infactíveis

Como a capacidade restante só diminui, um item que não coube uma vez nunca mais vai caber. Pode ser descartado em definitivo, sem testá-lo de novo.



Heurística construtiva

Implementação – problema da mochila (melhorias)

É **muito importante** que a implementação seja o mais eficiente possível. Abaixo são apresentadas algumas melhorias de eficiência da implementação.

Avaliação incremental

Manter o peso atual em uma variável, atualizada em $\mathcal{O}(1)$ a cada escolha, em vez de somar o vetor inteiro a cada chamada.

Ordenar os candidatos uma única vez

A ordem de qualidade entre os itens não muda, só o conjunto de candidatos que encolhe. Basta ordenar no início $[\mathcal{O}(n \log n)]$, não a cada passo.

Poda permanente de itens infactíveis

Como a capacidade restante só diminui, um item que não coube uma vez nunca mais vai caber. Pode ser descartado em definitivo, sem testá-lo de novo.

Resultado: redução na complexidade de aproximadamente $\mathcal{O}(n^2 \log n)$ para $\mathcal{O}(n \log n)$, mostrando *speedup* de $\sim 500\times$ em experimentos preliminares.



Heurística construtiva

Implementação – problema da mochila (melhorias)

```
1 def semi_greedy_construction(instance, k=3, seed=None):
2     rng = random.Random(seed)
3     solution = Solution(instance)
4     current_weight = 0
5     remaining = deque(sorted(range(instance.n), key=lambda i: instance.values[i], reverse=True))
6
7     while remaining:
8         rcl = []
9         while remaining and len(rcl) < k:
10             i = remaining.popleft()
11             if instance.weights[i] <= instance.capacity - current_weight:
12                 rcl.append(i)
13
14         if not rcl: break
15         chosen = rcl.pop(rng.randrange(len(rcl)))
16         for i in reversed(rcl):
17             remaining.appendleft(i)
18
19         solution.flip(chosen)
20         current_weight += instance.weights[chosen]
21
22     return solution
```

Heurística construtiva

Base para estratégias mais sofisticadas



Construção repetida: repete a construção (semi-gulosa) várias vezes e mantém a melhor solução encontrada.

GRASP: construção múltipla onde cada solução construída passa por uma busca local.

Iterated greedy: repetidamente **destrói** parte da solução (e.g., remove alguns elementos) e **reconstrói** com a mesma heurística construtiva.

Beam search: generaliza a construção para manter várias soluções parciais promissoras simultaneamente, em vez de uma única.

Decodificadores para metaheurísticas: a construção pode ser parametrizada por um vetor de entrada (e.g., chaves aleatórias), tornando-se um **decodificador** (e.g., base do BRKGA).

Fornecer solução inicial: qualquer metaheurística que precise de um ponto de partida (busca local, busca tabu, VNS, ...) pode usar a construção em vez de uma solução aleatória.

75POT – Pesquisa Operacional e Otimização
Prof. Marcelo de Souza